

Python 응용

모듈, 패키지 & 그래프 그리기

11-1. 모듈

11-1module.py

```
1 import theater_module
2 theater_module.price(3) # 3 명이 영화 보러 갔을 때 가격
3 theater_module.price_morning(4)
4 theater_module.price_soldier(5)
5
```

theater_module.py > ...

```
1 # 일반 가격
2 def price(people):
3     print("{0}명 가격은 {1}원 입니다.".format(people, people * 10000))
4
5 # 조조 할인 가격
6 def price_morning(people):
7     print("{0}명 조조 할인 가격은 {1}원 입니다.".format(people, people * 6000))
8
9 # 군인 할인 가격
10 def price_soldier(people):
11     print("{0}명 군인 할인 가격은 {1}원 입니다.".format(people, people * 4000))
```

11-2. 패키지

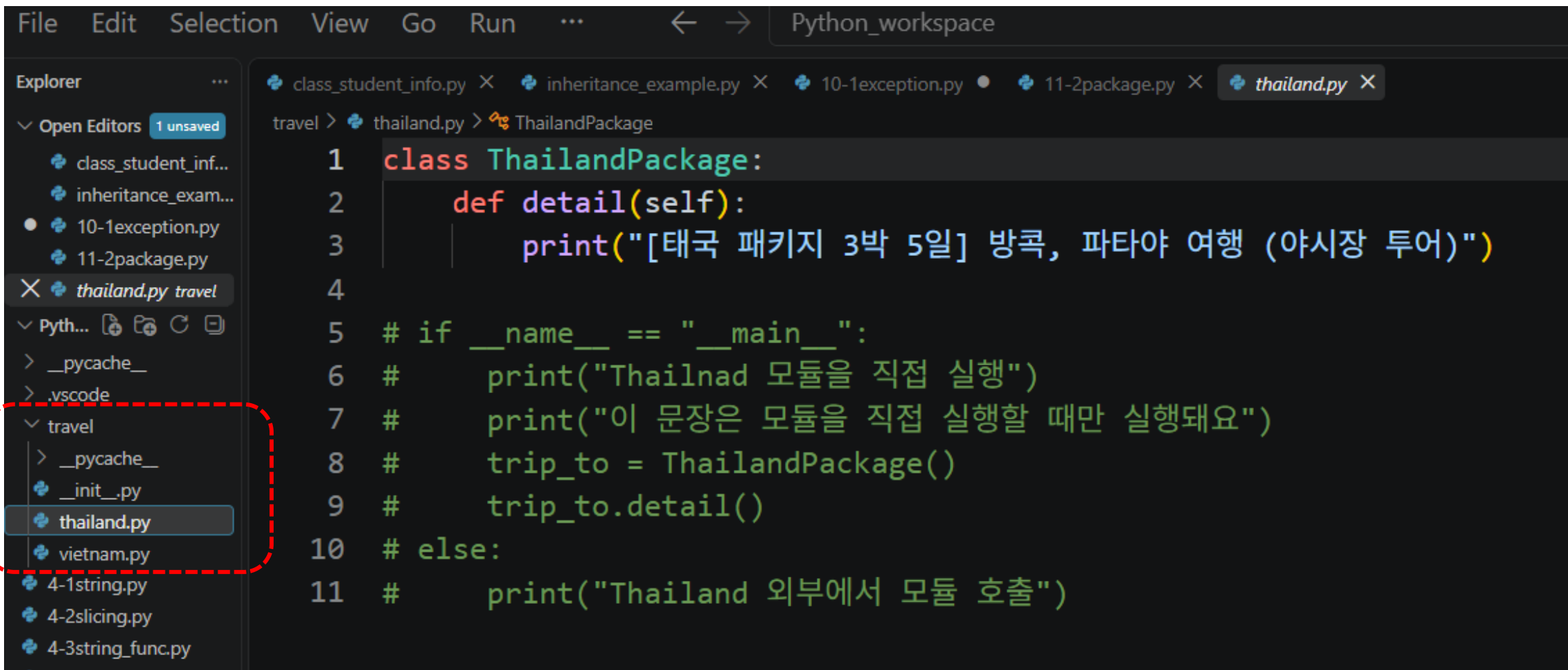
```
11-2package.py > ...
1 import travel.thailand
2 trip_to = travel.thailand.ThailandPackage()
3 trip_to.detail()
4
5 from travel.thailand import ThailandPackage
6 trip_to = ThailandPackage()
7 trip_to.detail()
8
9 from travel import vietnam
10 trip_to = vietnam.VietnaPackage()
11 trip_to.detail()
12
13
```

travel 이란 폴더를 만들어서 그 폴더 안에 Thailand.py, Vietnam.py 의 모듈을 만들어서 travel을 모듈이 여러 개 있는 패키지로 사용함

→ travel 패키지 내의 thailand 모듈을 import

→ travel 폴더내의 thailand 모듈 (thailand.py) 안에 있는 ThailandPackage 클래스를 실행

→ 이렇게 표현해도 실행이 된다



11-6.pip install

pip : package install path



The screenshot shows the PyPI website interface. At the top, there's a navigation bar with the PyPI logo, the text "pypi.org : python package index", and links for "Help", "Sponsors", and "Log in". The main content area has a large blue background with the text "Find, install and publish Python packages with the Python Package Index". Below this is a search bar labeled "Search projects" with a magnifying glass icon. Under the search bar, it says "Or [browse projects](#)". At the bottom of the main content area, there's a statistics bar showing "570,873 projects", "6,096,799 releases", "12,062,086 files", and "857,922 users". The footer contains the "python Package Index" logo and a description: "The Python Package Index (PyPI) is a repository of software for the Python programming language." It also includes a link: "PyPI helps you find and install software developed and shared by the Python community. [Learn about installing packages](#)".

pypi.org : python package index

Help Sponsors Log in

Find, install and publish Python packages
with the Python Package Index

Search projects

Or [browse projects](#)

570,873 projects 6,096,799 releases 12,062,086 files 857,922 users

pythonTM
Package
Index

The Python Package Index (PyPI) is a repository of software for the Python programming language.

PyPI helps you find and install software developed and shared by the Python community. [Learn about installing packages](#)



pandas



Help

Sponsors

Log in

Register

Filter by [classifier](#)

10,000+ projects for "pandas"

Order by

Relevance



Framework



Topic



Development Status



License



pandas 2.2.3

Powerful data structures for data analysis, time series, and statistics

Sep 20, 2024



pandas007 1.0.0



Search projects



pandas 2.2.3

`pip install pandas`



Copy to clipboard

data structures for data analysis, time series, and statistics

Project description



Project description



```
PS C:\Users\nices\OneDrive\바탕 화면\Python_workspace> pip install pandas
```

11-7.내장함수 11-8.외장함수

내장함수 : 이미 파이썬에 내장되어 있어서 import를 할 필요 없는 함수



list of python builtins

docs.python.org/3/library/functions.html

sh

3.12.6

3.12.6 Documentation » The Python Standard Library » Built-in Functions

Theme Auto

전체 이미지 쇼핑 동영상 뉴스



Python Docs

https://docs.python.org > library

Built-in Functions — Python 3.

Built-in Functions; A. abs(). aiter(). all(). bytearray(). bytes(). C. callable(). chr(). class

Sorting Techniques

GNU readline interface

Built-in Functions

The Python interpreter has a number of functions and types built into it that are always available. They are listed here in alphabetical order.

Built-in Functions

A

[abs\(\)](#)
[aiter\(\)](#)
[all\(\)](#)
[anext\(\)](#)
[any\(\)](#)
[ascii\(\)](#)

B

[bin\(\)](#)
[bool\(\)](#)
[breakpoint\(\)](#)
[bytearray\(\)](#)
[bytes\(\)](#)

E

[enumerate\(\)](#)
[eval\(\)](#)
[exec\(\)](#)

F

[filter\(\)](#)
[float\(\)](#)
[format\(\)](#)
[frozenset\(\)](#)

G

[getattr\(\)](#)
[globals\(\)](#)

L

[len\(\)](#)
[list\(\)](#)
[locals\(\)](#)

M

[map\(\)](#)
[max\(\)](#)
[memoryview\(\)](#)
[min\(\)](#)

N

[next\(\)](#)

R

[range\(\)](#)
[repr\(\)](#)
[reversed\(\)](#)
[round\(\)](#)

S

[set\(\)](#)
[setattr\(\)](#)
[slice\(\)](#)
[sorted\(\)](#)
[staticmethod\(\)](#)
[str\(\)](#)
[sum\(\)](#)

외장함수 : import를 해서 사용하는 함수

list of python modules



전체 이미지 쇼핑 동영상 뉴스 도서 지도 : 더보기



Python Docs

<https://docs.python.org> > py-modindex :

Python Module Index — Python 3.12.6 documentation

Python Module Index ; builtins, The **module** that provides interfaces for bzip2 compression and decompression. ;

Python Module Index

[_](#) | [a](#) | [b](#) | [c](#) | [d](#) | [e](#) | [f](#) | [g](#) | [h](#) | [i](#) | [j](#) | [k](#) | [l](#) | [m](#) | [n](#) | [o](#) | [p](#) | [q](#) | [r](#) | [s](#) | [t](#) | [u](#) | [v](#) | [w](#) | [x](#) | [z](#)

[__future__](#)

Future statement definitions

[__main__](#)

The environment where top-level code is run. Covers command-line interfaces, import-time behavior, and ``\_\_name\_\_ == '\_\_main\_\_'``.

Example: random

```
11-8external_fuc.py > ...
1  # glob : 경로내의 폴더 / 파일 목록 조회 (윈도우 dir)
2  # import glob
3  # print(glob.glob("*.py")) # 확장자가 py 인 모든 파일
4
5  # os : 운영체제에서 제공하는 기본 기능
6  # import os
7  # print(os.getcwd()) # 현재 디렉토리 표시
8
9  # folder = "sample_dir"
10
11 # if os.path.exists(folder):
12 #     print("이미 존재하는 폴더입니다.")
13 #     os.rmdir(folder)
14 #     print(folder, "폴더를 삭제하였습니다.")
15 # else:
16 #     os.makedirs(folder) # 폴더 생성
17 #     print(folder, "폴더를 생성하였습니다.")
18 # print(os.listdir())
19
20 # import time
21 # print(time.localtime())
22 # print(time.strftime("%Y-%m-%d %H:%M:%S"))
23
24 import datetime
25 # print("오늘 날짜는 ", datetime.date.today())
26 today = datetime.date.today() # 오늘 날짜 저장
27 td = datetime.timedelta(days=100) # 100일 저장
28 print("우리가 만난지 100일은", today + td) # 오늘부터 100일 후
```

cwd: current work directory)

Python 모듈, 패키지를 이용한
그래프 그리기

그래프 그리기

matplotlib 모듈을 이용한 그래프그리기

graph1

matplotlib 모듈 import

```
PS C:\Users\nices\OneDrive\바탕 화면\Python_workspace> pip install matplotlib
```

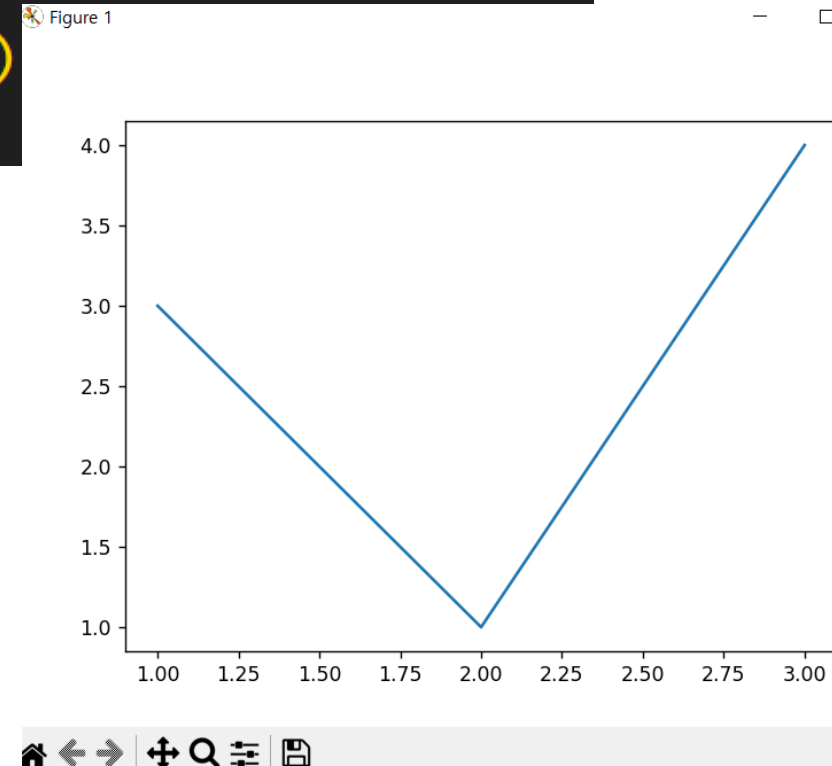
Command window에서
"pip install" 명령어 실행

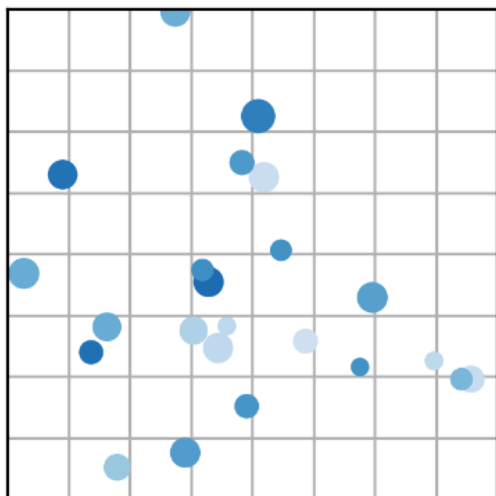
```
pip install matplotlib
pip install numpy
pip install scipy
```

```
C: > nicesw > lecture > 2026 > 2학기 > 전산물리학 > lecture 3 > graph1.py > ...
1 import matplotlib.pyplot as plt
2
3 x=[1, 2, 3]
4 y=[3, 1, 4]
5 z=[3, 2, 3]
6 plt.plot(x, y)
7 plt.show()
```

Anaconda powershell 에서
"conda install" 명령어 실행

```
conda install matplotlib
conda install numpy
conda install scipy
```





`scatter(x, y)`

Matplotlib: Visualization with Python

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations Python. Matplotlib makes easy things easy and hard things possible.

- Create [publication quality plots](#).
- Make [interactive figures](#) that can zoom, pan, update.
- Customize [visual style](#) and [layout](#).
- Export to [many file formats](#).
- Embed in [JupyterLab](#) and [Graphical User Interfaces](#).
- Use a rich array of [third-party packages](#) built on Matplotlib.

[Try Matplotlib \(on Binder\)](#)



[Getting Started](#)



[Examples](#)



[Reference](#)



[Cheat Sheets](#)



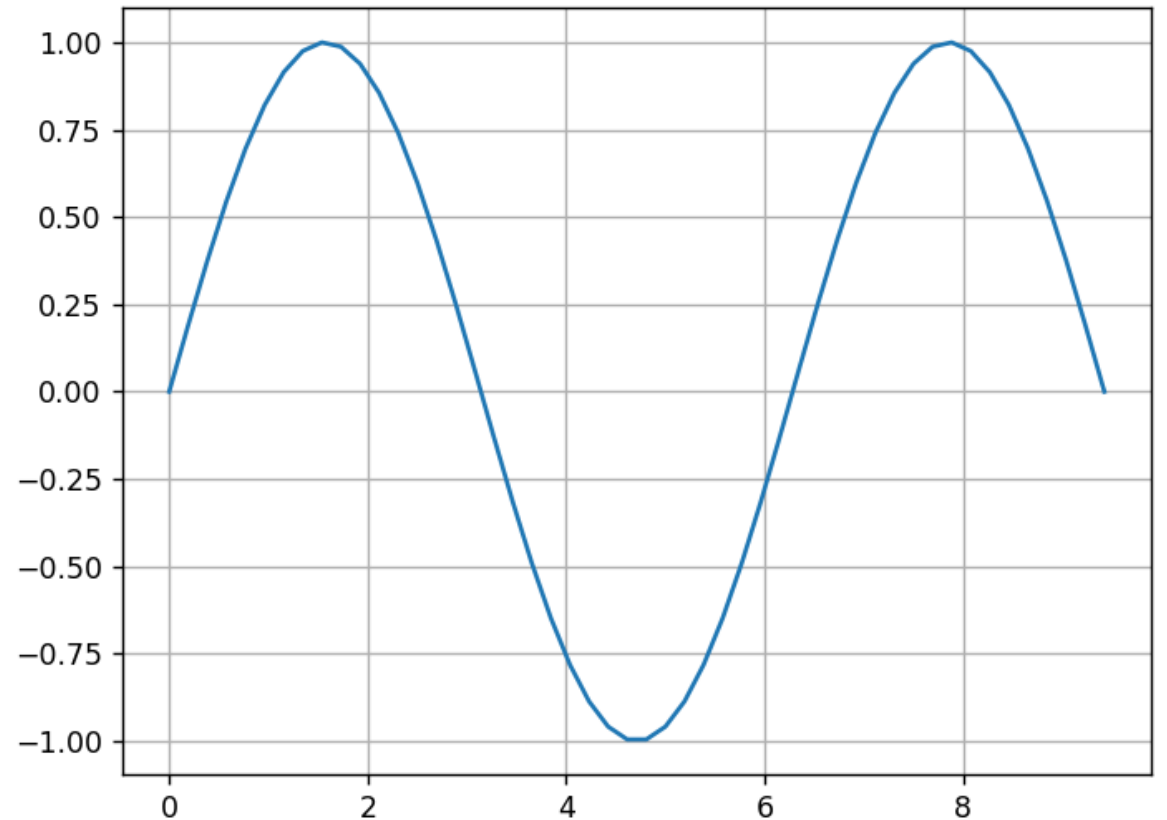
[Documentation](#)

<https://matplotlib.org/>

graph2

C: > nicesw > lecture > 2026 > 2학기 > 전산물리학 > lecture 3 > graph2.py > ...

```
1  """함수 그래프 그리기
2  sin(x) 함수를 지정한 범위 [0, 3Pi]
3  """
4  import numpy as np
5  import matplotlib.pyplot as plt
6
7
8  def func(x):
9      return np.sin(x)
10
11
12  if __name__ == "__main__":
13      x = np.linspace(0, np.pi * 3)
14      plt.plot(x, func(x))
15      plt.grid(True)
16      plt.show()
```



여러종류의 함수를 그리기

- $\cos(x)$, x 는 $[-\pi, +2\pi]$
 - numpy 활용: `np.cos(x)`
- $3x + 2$, x 는 $[0, 5]$
 - `3 * x + 2`
- $\ln(x)$, x 는 $[0, 10]$
 - numpy 활용: `np.log(x)`
- e^x , x 는 $[-5, 5]$
 - numpy 활용: `np.exp(x)`

C: > nicesw > lecture > 2026 > 2학기 > 전산물리학 > lecture 3 > graph3_examples.py > ...

```
1  #y=3x+2 그래프 그리기
2  #함수를 지정한 범위 [0, 10]의 그래프도
3
4  import numpy as np
5  import matplotlib.pyplot as plt
6
7  # def func(x):
8  #     return np.sin(x)
9
10 def func(x):
11     return x
12
13 if __name__ == "__main__":
14     x = np.linspace(0, 10)
15     y = 3 * x + 2
16     plt.plot(x, y)
17     #x = np.linspace(0, np.pi * 3)
18     #plt.plot(x, func(x))
19     plt.grid(True)
20     plt.show()
```

데이터 파일을 그래프로 그리기

The `os` module in Python provides functions for interacting with the operating system. It allows you to perform tasks such as working with directories and files, retrieving environmental variables, and running system commands. Here's a breakdown of some key functionalities of the `os` module:

1. Directory and Path Operations

- `os.chdir(path)` : Changes the current working directory to the specified path.
- `os.getcwd()` : Returns the current working directory.
- `os.listdir(path)` : Lists all files and directories in the given directory.
- `os.path.abspath(path)` : Returns the absolute path of the specified file or directory.
- `os.path.dirname(path)` : Returns the directory name of the given path.
- `os.mkdir(path)` : Creates a new directory.
- `os.rmdir(path)` : Removes an empty directory.

read from and write to files located in the same folder as your script without needing to specify their full paths

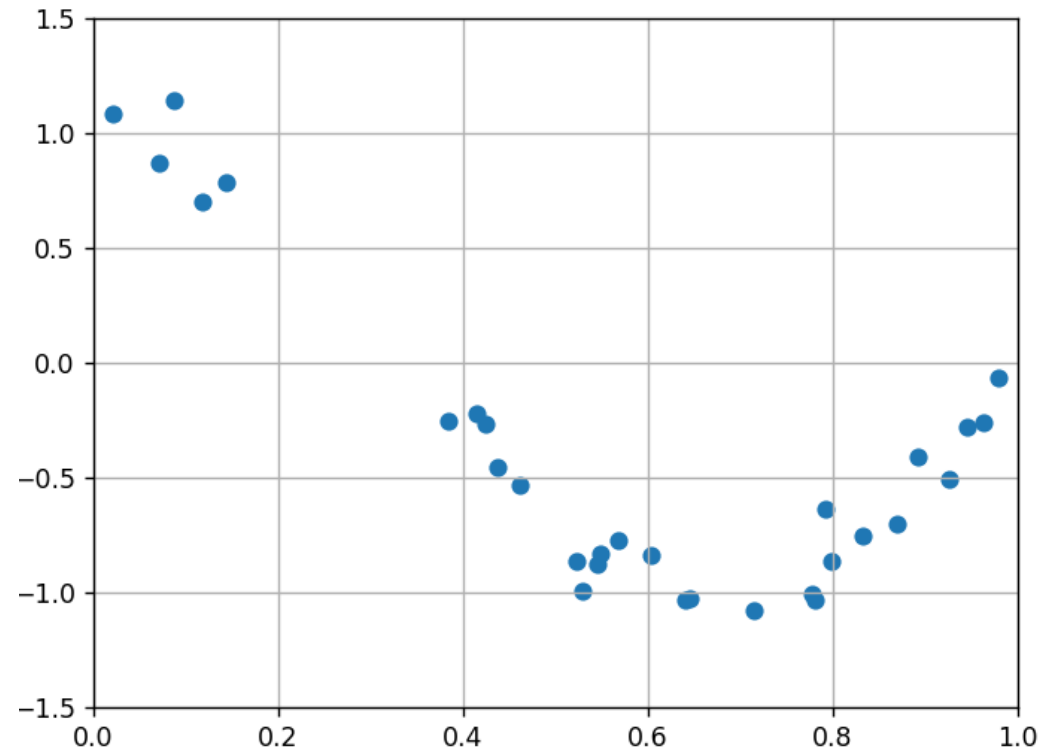
```
C: > nicesw > lecture > 2026 > 2학기 > 전산물리학 > lecture 3 > graph4_from_data1.py
1  # 파일의 데이터로 그래프 그리기
2
3  # input.csv로부터 데이터를 읽어들이어서 그래프로 출력
4
5  import os
6
7  os.chdir(os.path.abspath(os.path.dirname(__file__)))
8
9  #소스코드의 실행 디렉토리와 데이터파일의 디렉토리가 일치하도록 조정
10
11 import matplotlib.pyplot as plt
12
13 x = []
14 y = []
15
16 with open("input.csv", "r") as f:
17     for line in f.readlines():
18         items = line.split(" ")
19         x.append(float(items[0]))
20         y.append(float(items[1]))
21
22 if __name__ == "__main__":
23     plt.scatter(x, y)
24     plt.xlim(0, 1)
25     plt.ylim(-1.5, 1.5)
26     plt.grid(True)
27     # plt.savefig("data_plot_1.png", bbox_inches="tight")
28     plt.show()
29
```

https://github.com/nicesw77/com_phys2024/blob/main/input.csv

```
x = []
y = []

with open("input.csv", "r") as f:
    for line in f.readlines():
        items = line.split(" ")
        x.append(float(items[0]))
        y.append(float(items[1]))
```

```
if __name__ == "__main__":
    plt.scatter(x, y)
    plt.xlim(0, 1)
    plt.ylim(-1.5, 1.5)
    plt.grid(True)
    # plt.savefig("data_plot_1.png", bbox_inches="tight")
    plt.show()
```

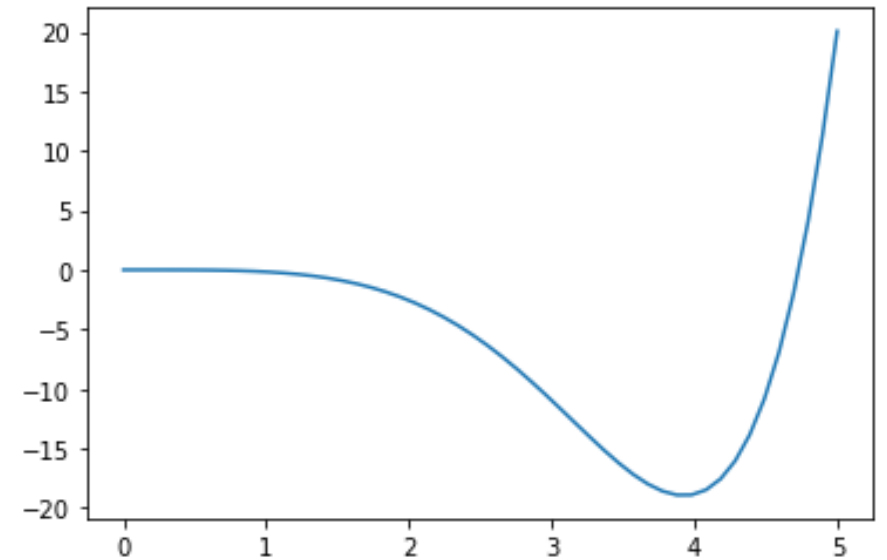


해석 함수(analytical function)의 곡선 그리기

$$f(x) = \cos(x)\cosh(x) - 1 \quad 0 \leq x \leq 5$$

C: > nicesw > lecture > 2026 > 2학기 > 전산물리학 > lecture 3 > matplotlib_eq1.py >

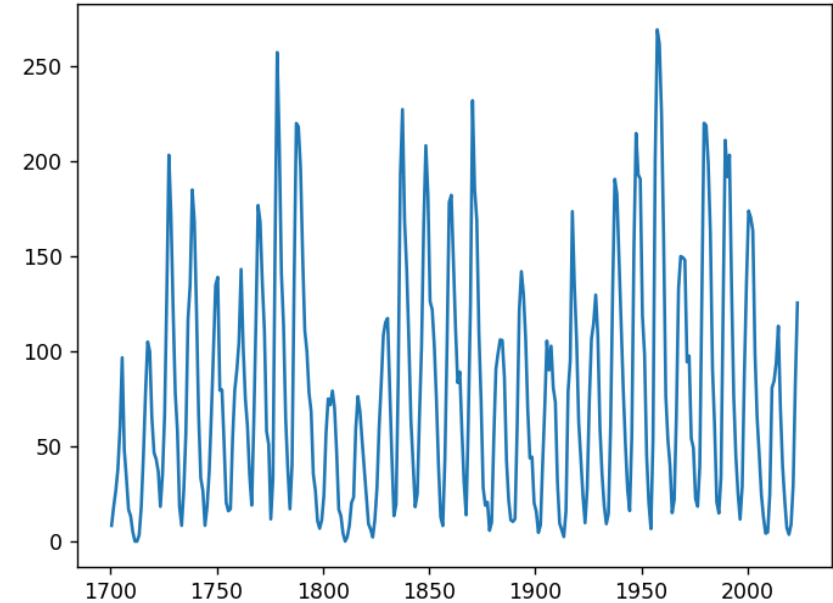
```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(0, 5)
5 f = lambda x: np.cos(x)*np.cosh(x)-1
6 plt.plot(x, f(x))
7 plt.show()
```



데이터를 다운로드해서 그래프 그리기

1848년에 요한 루돌프 볼프(Johann Rudolph Wolf)는 태양 표면의 개별 흑점과 흑점 그룹의 수를 세어 태양 활동을 정량화하는 방법을 고안했다.

- 그는 1년 동안 흑점 그룹 수에 10을 곱하고 개별 흑점은 총 수를 더하여 현재 볼프 흑점 수(Wolf sunspot number)라고 부르는 값을 계산했다.
- 이 데이터는 <http://www.sidc.be/silso/datafiles>에서 .csv 형식인 파일 이름 SN_y_tot_V2.0.csv로 다운로드 할 수 있다.

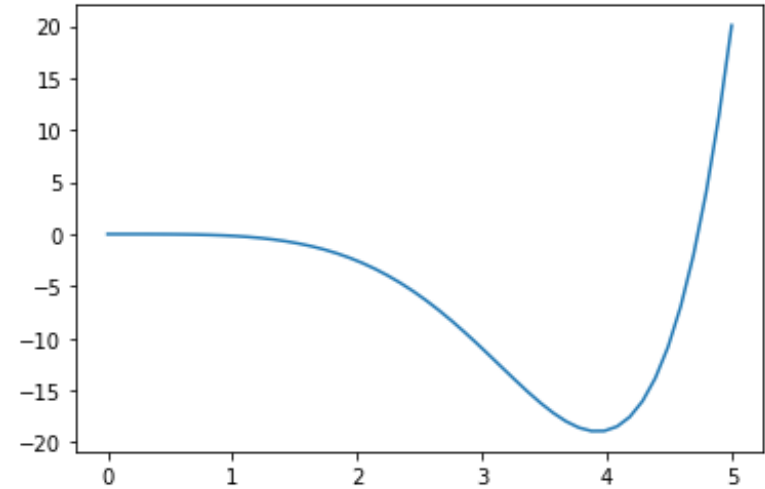


C: > nicesw > lecture > 2026 > 2학기 > 전산물리학 > lecture 3 > sun_spot_plot.py > ...

눈금선(grid), 축 및 플롯 제목 넣기

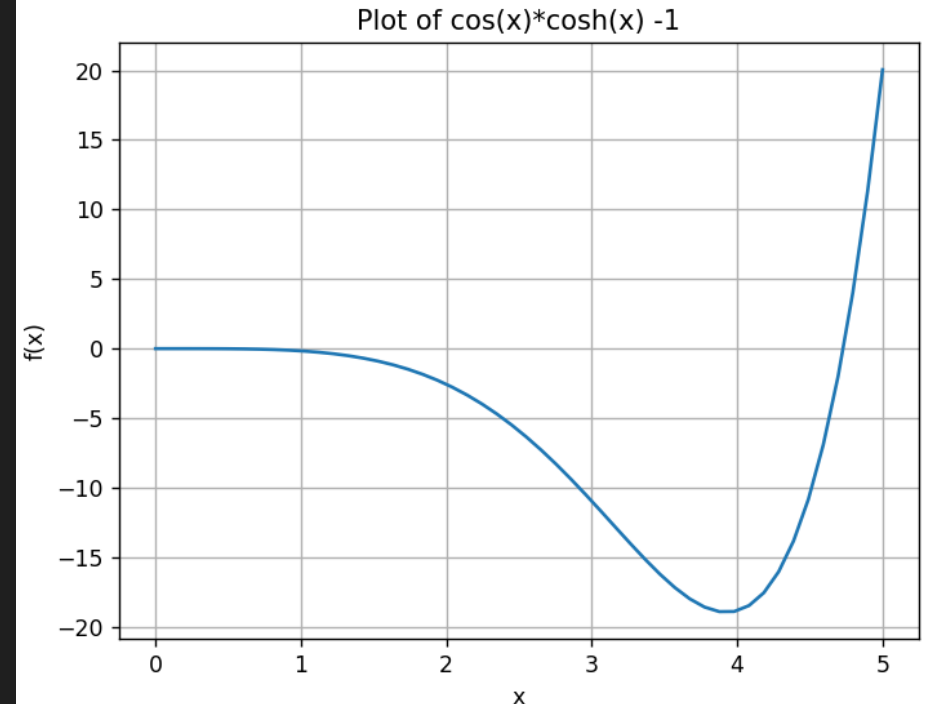
C: > nicesw > lecture > 2026 > 2학기 > 전산물리학 > lecture 3 > matplotlib_eq1.py >

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(0, 5)
5 f = lambda x: np.cos(x)*np.cosh(x)-1
6 plt.plot(x, f(x))
7 plt.show()
```



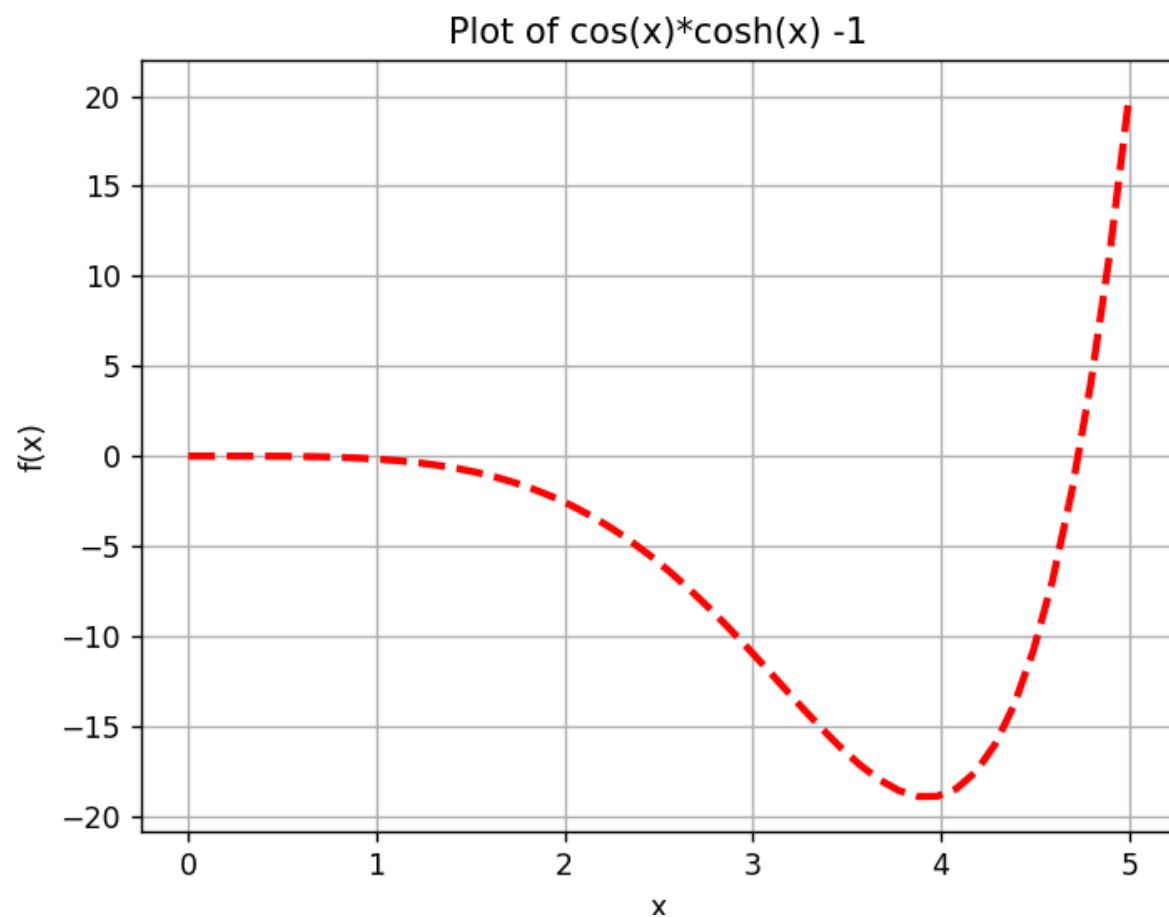
C: > nicesw > lecture > 2026 > 2학기 > 전산물리학 > lecture 3 > matplotlib_eq1.py >

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(0, 5)
5 f = lambda x: np.cos(x)*np.cosh(x)-1
6 plt.plot(x, f(x))
7
8 plt.grid()
9 plt.xlabel('x')
10 plt.ylabel('f(x)')
11 plt.title('Plot of cos(x)*cosh(x) -1')
12 plt.show()
```



색상과 선 스타일 지정

```
6 plt.plot(x, f(x), color = 'r', linestyle = '--', linewidth = '2.5')
```



문자	색상
k	검정색
b	청색
g	녹색
r	붉은색
c	청록색
m	자홍색
y	노란색
w	흰색

문자	스타일
-	실선
--	파선
:	점선
-.	파-점선

Scatter 그래프에서 마커 스타일

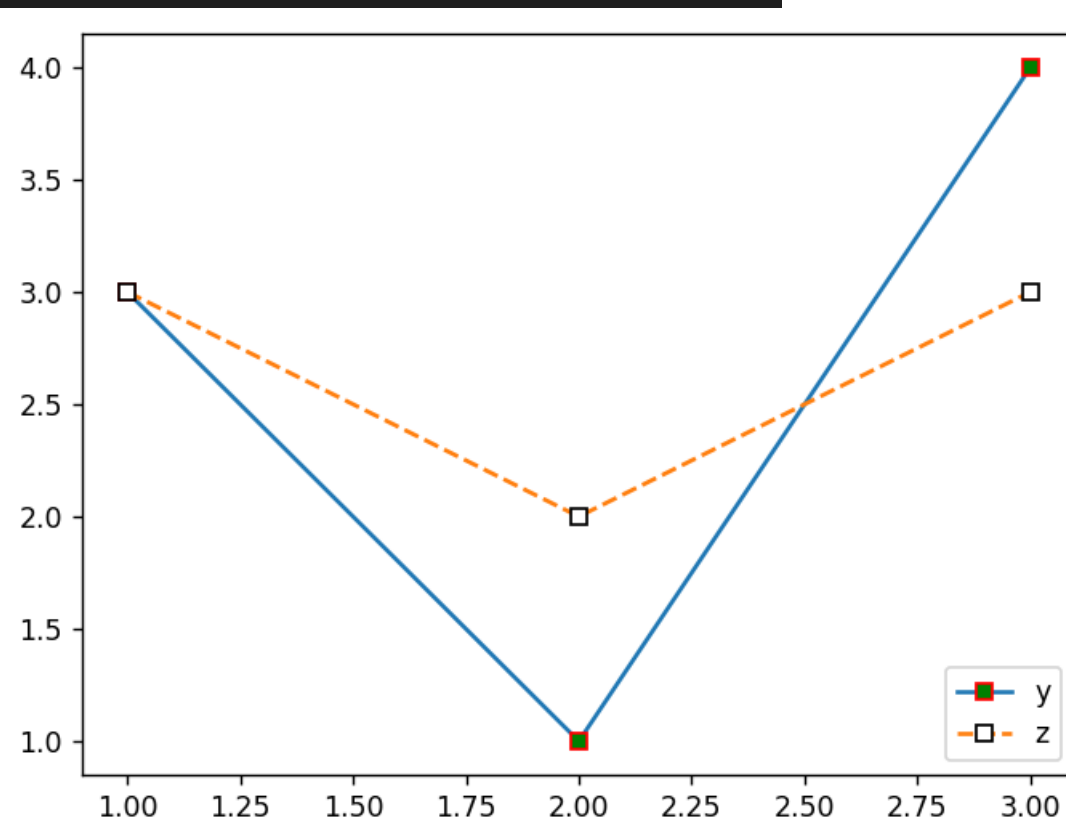
```
1 import matplotlib.pyplot as plt
2
3 x=[1, 2, 3]
4 y=[3, 1, 4]
5 plt.scatter(x, y, marker='s', c='b', edgecolors='r')
6 plt.show()
```

```
plt.plot(x, y, marker='s', markeredgecolor='r', markerfacecolor='g')
plt.show()
```

문자	기호	설명
.	•	점
o	○	원
+	+	플러스 사인
x	×	
D	◇	다이아몬드
v	▽	델
^	△	삼각형
s	□	사각형

C: > nicesw > lecture > 2026 > 2학기 > 전산물리학 > lecture 3 > graph1.py > ...

```
1  import matplotlib.pyplot as plt
2
3  x=[1, 2, 3]
4  y=[3, 1, 4]
5  z=[3, 2, 3]
6  # plt.scatter(x, y, marker='s', c='b', edgecolors='r')
7  plt.plot(x, y, marker='s', markeredgecolor='r', markerfacecolor='g', label='y')
8  plt.plot(x, z, marker='s', ls='--', mec='k', mfc='w', label='z')
9  plt.legend(loc='lower right')
10 plt.show()
11
```



데이터

API

기후통계분석

간행물

소통과 참여

기상현상증명



데이터

기상관측



지상



- 종관기상관측(ASOS)

Home > 데이터 > 기상관측 > 지상 > 종관기상관측(ASOS)

종관기상관측(ASOS) - 자료

■ 자료설명

자료설명

종관기상관측이란 종관규모의 날씨를 파악하기 위하여 정해진 시각에 모든 관측소에서 같은 시각에 실시하는 지상관측을 말합니다. 종관규모는 일기도에 표현되어 있는 보통의 고기압이나 저기압의 공간적 크기 및 수명을 말하며, 주로 매일의 날씨 현상을 뜻합니다.

자료형태

분, 시간(매정시), 일, 월, 연

제공기간

1904년~(지점별, 요소별 다름)

기상청 기온 데이터를 통해 2026년 서울의 9월 평균기온을 다운로드하여 그래프로 출력하기

Q & A